Blueprint: what the Lean development proves

Generated by `scripts/blueprint.py`

August 19, 2026

Generated from the formalization, not written alongside it. Dependencies are read from Lean's own printed proof terms; the classification is propagated from the imported axioms and agrees with `#print axioms`. Signatures are transliterated to ASCII, and the generator aborts on any character it has no mapping for rather than substituting a placeholder.

[imported] assumed. Both are classical theorems, neither open nor near the frontier this development concerns.
[rests on an import] proved, but its proof reaches an imported axiom.
[proved outright] proved from nothing beyond Lean's own three axioms.

Of 57 declarations, 45 are proved outright, 10 rest on an import, and 2 are imported. No `sorry` occurs anywhere.

1 Imported results

`Diaz/Axioms.lean`

`exists_ringHom_of_transcendental` [imported]

Steinitz extension. If ‘u‘ and ‘t‘ are both transcendental over a subfield ‘K‘ of ‘C‘, some ring endomorphism of ‘C‘ fixes ‘K‘ pointwise and sends ‘u‘ to ‘t‘. Why it is true: ‘u |-> t‘ is an isomorphism ‘K(u) -> K(t)‘ fixing ‘K‘, because both are transcendental, so both are rational function fields over ‘K‘. Extend a transcendence basis of ‘C‘ over ‘K(u)‘ into one over ‘K(t)‘ — both have the cardinality of the continuum — and then extend algebraically, ‘C‘ being algebraically closed. See Lang, *Algebra*, 3rd ed., Ch. VIII, or Steinitz's theorem in any account of field theory. The thesis is deliberately weak: a ring *endomorphism*, not an automorphism, which is all the results below consume, and which is automatically injective since ‘C‘ is a field. The hypothesis is necessary — for ‘u‘ algebraic over ‘K‘ and ‘t‘ not, no such map exists. Mathlib has the ingredients (‘IsAlgClosed.equivOfTranscendenceBasis‘, ‘IsAlgClosed.lift‘) but not the assembled statement.

```
axiom exists_ringHom_of_transcendental {K : Subfield C} {u t : C}
 (hu : Transcendental (K) u) (ht : Transcendental (K) t) :
 exists Phi : C ->+* C, (forall a in K, Phi a = a) and Phi u = t

end Diaz
```

`hermite_lindemann` [imported]

Hermite–Lindemann. If ‘u != 0‘ and ‘exp u‘ is algebraic, then ‘u‘ is transcendental. This is the contrapositive of the usual statement, that a non-zero algebraic number has transcendental exponential. See A. Baker, *Transcendental Number Theory*, Cambridge University Press 1975, Theorem 1.4. **This will become dischargeable.** Mathlib master carries only the analytic half (‘NumberTheory/Transcendental/Lindemann/AnalyticalPart‘), but the theorem itself is

formalized in the open PR ‘leanprover-community/mathlib4#28013‘, as theorem transcendental_exp {a : C} (a0 : a != 0) (ha : IsAlgebraic Z a) : Transcendental Z (exp a) which is the contrapositive of this axiom, over ‘Z‘ rather than ‘Q‘ – the same condition in characteristic zero. When that PR merges, delete this axiom and derive it; the statement below is shaped to make that a local change.

```
axiom hermite_lindemann {u : C} (hu : u != 0)
 (hexp : IsAlgebraic Q (Complex.exp u)) : Transcendental Q u

/-









 -/
```

2 The closure theorem

`Diaz/Closure.lean`

`conj_eq_rho_div` [proved outright]

With ‘rho = u * conj u‘, the conjugate of ‘u‘ is ‘rho / u‘. Trivial as algebra, and it is the entire content of the closure theorem: ‘conj u‘ is not an independent quantity but a rational function of ‘u‘ over the base field. Everything else follows.

```
theorem conj_eq_rho_div (hu : u != 0) : (u * conj u) / u = conj u
```

`conj_mem_hull` [proved outright]

Complex conjugation maps ‘hull K u‘ into itself, given that ‘K‘ is conjugation-stable and contains ‘rho = u * conj u‘.

```
theorem conj_mem_hull (hK : forall z in K, conj z in K) (hu : u != 0)
 (hrho : u * conj u in K) :
 forall z in hull K u, conj z in hull K u
```

Uses: `conj_eq_rho_div`, `mem_hull_of_mem_base`, `self_mem_hull`

`conj_not_linear` [proved outright]

Conjugation is **not** ‘K‘-linear once ‘K‘ contains a non-real number. A ‘K‘-linear map ‘f‘ would satisfy ‘f (a * z) = a * f z‘ for ‘a in K‘; taking ‘z = 1‘ forces ‘conj a = a‘, so only a totally real base field could carry conjugation as a linear map. Since the intended base is the algebraic numbers, which contain ‘i‘, conjugation on the hull is a ring involution and nothing stronger.

```
theorem conj_not_linear {a : C} (ha : conj a != a) :
 not (forall z : C, conj (a * z) = a * conj z)
```

conj_not_linear_hull [proved outright]

The form actually needed: conjugation is not ‘K‘-linear *on the hull*. The refuted claim is about the hull, and ‘forall z in hull‘ is weaker than ‘forall z : C‘, so its negation is the stronger statement. It is free: the witness ‘z = 1‘ lies in the hull.

```
theorem conj_not_linear_hull {a : C} (hne : conj a != a) :
 not (forall z in hull K u, conj (a * z) = a * conj z)
```

Uses: hull

conj_not_linear_of_I [proved outright]

Concretely: conjugation is not linear over any base containing ‘i‘.

```
theorem conj_not_linear_of_I :
 not (forall z : C, conj (Complex.I * z) = Complex.I * conj z)
```

Uses: conj_not_linear

eqOn_hull [proved outright]

Two ring homomorphisms of ‘C‘ agreeing on ‘K‘ and at ‘u‘ agree on the whole hull. This is the "a homomorphism is determined by these data" step of the closure theorem’s proof.

```
theorem eqOn_hull (f g : C ->+* C) (hK : forall z in K, f z = g z) (hu : f u = g u) :
 forall z in hull K u, f z = g z
```

Uses: hull

hull [proved outright]

The subfield of ‘C‘ generated by ‘K‘ together with ‘u‘.

```
noncomputable def hull : Subfield C
```

mem_hull_of_mem_base [proved outright]

```
theorem mem_hull_of_mem_base {z : C} (hz : z in K) : z in hull K u
```

Uses: hull

ne_zero_of_notMem [proved outright]

An element outside a subfield is non-zero, subfields containing ‘0‘.

```
theorem ne_zero_of_notMem {K : Subfield C} {z : C} (h : z notin K) : z != 0
```

self_mem_hull [proved outright]

```
theorem self_mem_hull : u in hull K u
```

Uses: hull

transcendental_candidate_over_base [rests on an import]

The bridge, assembled: a candidate is transcendental over the base.

```
theorem transcendental_candidate_over_base {L : Subfield C}
 [Algebra.IsAlgebraic Q (L)] (hu : u != 0)
 (hexp : IsAlgebraic Q (Complex.exp u)) : Transcendental (L) u
```

Uses: transcendental_of_base, transcendental_of_candidate

`transcendental_ne_zero` [proved outright]

A transcendental element is non-zero: ‘0‘ is a root of ‘X‘.

```
theorem transcendental_ne_zero {F : Type*} [Field F] [Algebra F C] {z : C}
 (h : Transcendental F z) : z != 0
```

`transcendental_of_base` [proved outright]

Transcendence over ‘Q‘ upgrades to transcendence over any base algebraic over ‘Q‘ — in particular over the algebraic numbers, which is the intended base. Without this the imported axiom is a dead leaf: the results above take transcendence over the base as a hypothesis, while Hermite–Lindemann supplies it only over ‘Q‘. The algebraicity hypothesis is necessary rather than decorative: for a base containing ‘u‘ the conclusion is false.

```
theorem transcendental_of_base {L : Subfield C} [Algebra.IsAlgebraic Q (L)]
 {z : C} (h : Transcendental Q z) : Transcendental (L) z
```

`transcendental_of_candidate` [rests on an import]

A counterexample to Diaz’s conjecture is transcendental. The only imported input is Hermite–Lindemann.

```
theorem transcendental_of_candidate (hu : u != 0)
 (hexp : IsAlgebraic Q (Complex.exp u)) : Transcendental Q u
```

Uses: `hermite_lindemann`

3 The model

`Diaz/Model.lean`

`conj_coords` [proved outright]

Complex conjugation acts on the plane by swapping coordinates. This is what licenses stating the involution in coordinates.

```
theorem conj_coords (t : C) (a b : Q) :
 conj ((a : C) * t + (b : C) * conj t) = (b : C) * t + (a : C) * conj t
```

`exists_transcendental_on_circle` [rests on an import]

A transcendental point on the circle. For any non-zero ‘r‘ in a base ‘K‘ algebraic over ‘Q‘, the number ‘t = r * eˆi‘ is transcendental over ‘K‘, with ‘t * conj t‘ equal to ‘r * conj r‘ **and** in the base. Both conclusions are needed: the model lemmas consume the membership, while the transfer results consume the equality ‘t * conj t = u * conj u‘. An earlier version gave only the membership, so no exhibited ‘t‘ could discharge the transfer hypotheses. The hypothesis ‘conj r in L‘ is needed and is not implied by ‘r in L‘: for ‘L = Q(2ˆ{1/3}omega)‘ and ‘r = 2ˆ{1/3}omega‘ one has ‘r * conj r = 2ˆ{2/3} notin L‘. Every result that consumes this already assumes ‘L‘ conjugation-stable. So the hypotheses of ‘norm_mem_iff‘ are not merely satisfiable in the abstract; they are met by an explicit complex number.

```
theorem exists_transcendental_on_circle {L : Subfield C}
 [Algebra.IsAlgebraic Q (L)] {r : C} (hr : r in L) (hrc : conj r in L)
 (hr0 : r != 0) :
 exists t : C, t != 0 and Transcendental (L) t and t * conj t = r * conj r
 and t * conj t in L
```

Uses: `transcendental_exp_I`, `transcendental_of_base`

exists_transcendental_on_circle_sq [rests on an import]

The circle condition in the form the matrix results consume. ‘exists_transcendental_on_circle‘ gives ‘t * conj t = r * conj r‘, while ‘Rigidity‘ states its hypotheses as ‘u * conj u = r ^ 2‘. These agree exactly when ‘r‘ is real — the intended case, ‘r = sqrt rho‘ with ‘rho = |u|^2‘ — which is not implied by ‘r in L‘ and so has to be said.

```
theorem exists_transcendental_on_circle_sq {L : Subfield C}
 [Algebra.IsAlgebraic Q (L)] {r : C} (hr : r in L) (hrr : conj r = r)
 (hr0 : r != 0) :
 exists t : C, t != 0 and Transcendental (L) t and t * conj t = r ^ 2
 and t * conj t in L
```

Uses: exists_transcendental_on_circle

indep [proved outright]

‘t‘ and ‘conj t‘ are independent over ‘Q‘. This is what makes coordinates ‘(a, b) |-> a t + b conj t‘ well defined, so that a function of the coordinates is a function on the plane. It is the analogue of the axis lemma.

```
theorem indep (hT : Transcendental K t) (hrho : t * conj t in K)
 {a b : Q} (h : (a : C) * t + (b : C) * conj t = 0) : a = 0 and b = 0
```

Uses: conj_eq_rho_div, sq_add_sq_notMem, transcendental_ne_zero

norm_form [proved outright]

On the rational plane spanned by ‘t‘ and ‘conj t‘, (a t + b tbar)(a tbar + b t) = (a^2 + b^2)*t tbar + a b*(t^2 + tbar^2). This is ‘x sigma(x) = (a^2 + b^2) rho + a b (T^2 + rho^2/T^2)‘ of ‘prop:model‘, with ‘conj‘ in place of ‘sigma‘.

```
theorem norm_form (t : C) (a b : Q) :
 ((a : C) * t + (b : C) * conj t) * conj ((a : C) * t + (b : C) * conj t)
 = ((a : C) ^ 2 + (b : C) ^ 2) * (t * conj t)
 + ((a : C) * (b : C)) * (t ^ 2 + (conj t) ^ 2)
```

norm_mem_iff [proved outright]

The characterisation of ‘D0‘. For ‘a, b‘ rational, the element ‘x = a t + b conj t‘ has ‘x * conj x in K‘ exactly when ‘a = 0‘ or ‘b = 0‘. So the elements of the rational plane with "algebraic modulus" are precisely the rational multiples of ‘t‘ and of ‘conj t‘ — the two rays of ‘prop:model‘, and in particular there are non-zero ones. Every algebraic constraint a Diaz candidate satisfies is satisfied here too.

```
theorem norm_mem_iff (hT : Transcendental K t) (hrho : t * conj t in K) (a b : Q) :
 ((a : C) * t + (b : C) * conj t) * conj ((a : C) * t + (b : C) * conj t) in K
 <-> a = 0 or b = 0
```

Uses: norm_form, sq_add_sq_notMem

sq_add_sq_notMem [proved outright]

If ‘t‘ is transcendental over ‘K‘ and ‘rho = t * conj t‘ lies in ‘K‘, then the cross term ‘t^2 + conj(t)^2‘ does not lie in ‘K‘. Otherwise ‘t‘ would satisfy ‘X^4 - c X^2 + rho^2‘, a non-zero polynomial over ‘K‘. This is what forces ‘a b = 0‘ in the model.

```
theorem sq_add_sq_notMem (hT : Transcendental K t) (hrho : t * conj t in K) :
 t ^ 2 + (conj t) ^ 2 notin K
```

`transcendental_exp_I` [rests on an import]

‘e^i‘ is transcendental over ‘Q‘. Immediate from Hermite–Lindemann, since ‘i‘ is algebraic and non-zero.

```
theorem transcendental_exp_I : Transcendental Q (Complex.exp Complex.I)
```

Uses: `transcendental_of_candidate`

4 The formal exponential

`Diaz/Exponential.lean`

`Exp0` [proved outright]

‘Exp0 (a, b) = 2 ^ (a + b)‘, the formal exponential of the model.

```
noncomputable def Exp0 (x : Q x Q) : R
```

`Exp0_add` [proved outright]

‘Exp0‘ is a homomorphism from the plane to ‘R^ x ‘.

```
theorem Exp0_add (x y : Q x Q) : Exp0 (x + y) = Exp0 x * Exp0 y
```

Uses: `Exp0`

`Exp0_eq_one_iff` [proved outright]

The kernel of this ‘Exp0‘ is the divisible line ‘a + b = 0‘. A property of this choice, not of the model — see the caveat in the file header, where a variant with a lattice kernel is given.

```
theorem Exp0_eq_one_iff (x : Q x Q) : Exp0 x = 1 <-> x.1 + x.2 = 0
```

Uses: `Exp0`, `two_rpow_eq_one_iff`

`Exp0_isAlgebraic` [proved outright]

Every value of ‘Exp0‘ is algebraic: the model really is a model of the *arithmetic* situation, not merely of the field structure.

```
theorem Exp0_isAlgebraic (x : Q x Q) : IsAlgebraic Q (Exp0 x)
```

Uses: `Exp0`, `isAlgebraic_two_rpow`

`Exp0_ne_zero` [proved outright]

```
theorem Exp0_ne_zero (x : Q x Q) : Exp0 x != 0
```

Uses: `Exp0_pos`

`Exp0_pos` [proved outright]

```
theorem Exp0_pos (x : Q x Q) : 0 < Exp0 x
```

Uses: `Exp0`

`Exp0_pow_eq_one_iff` [proved outright]

The image of this ‘Exp0‘ is torsion-free. A standard kernel ‘Zomega‘ makes ‘Exp (pomega/m)‘ a primitive ‘m‘-th root of unity, so the image would have to contain every root of unity; ‘2^Q‘ contains none but ‘1‘. Again a property of this choice: the variant in the file header has both a lattice kernel and torsion in its image.

```
theorem Exp0_pow_eq_one_iff (x : Q x Q) {n : N} (hn : n != 0) :
 Exp0 x ^ n = 1 <-> Exp0 x = 1
```

Uses: `Exp0_eq_one_iff`

`Exp0_swap` [proved outright]

‘Exp0‘ commutes with the involution. In coordinates the involution swaps ‘a‘ and ‘b‘; the values are real, so complex conjugation fixes them, and ‘a + b‘ is symmetric.

```
theorem Exp0_swap (x : Q x Q) : Exp0 (x.2, x.1) = Exp0 x
```

Uses: `Exp0`

`Exp0_swap_conj` [proved outright]

The form the model actually needs: ‘Exp0 . sigma = conj . Exp0‘, stated with the conjugation rather than only with the coordinate swap.

```
theorem Exp0_swap_conj (x : Q x Q) :
 ((Exp0 (x.2, x.1) : C)) = (starRingEnd C) ((Exp0 x : C))
```

Uses: `Exp0_swap`

`isAlgebraic_two_rpow` [proved outright]

‘2 ^ q‘ is algebraic for every rational ‘q‘: it is a root of ‘X ^ q.den - 2 ^ q.num‘.

```
theorem isAlgebraic_two_rpow (q : Q) : IsAlgebraic Q ((2 : R) ^ (q : R))
```

`model_falsifies` [proved outright]

The analogue of Diaz’s conjecture is false in the model. There is a non-zero element of the plane whose product with its conjugate lies in ‘K‘ and which carries an algebraic formal exponential. This is the claim the closure theorem is pointed at, and it is what makes the absence of a contradiction more than a failure to find one.

```
theorem model_falsifies {K : Subfield C} {t : C}
 (hrho : t * conj t in K) (ht0 : t != 0) :
 exists a b : Q, ((a : C) * t + (b : C) * conj t) != 0
 and IsAlgebraic Q (Exp0 (a, b))
 and ((a : C) * t + (b : C) * conj t)
 * conj ((a : C) * t + (b : C) * conj t) in K
```

Uses: `Exp0_isAlgebraic`

`two_rpow_eq_one_iff` [proved outright]

```
theorem two_rpow_eq_one_iff (r : R) : (2 : R) ^ r = 1 <-> r = 0
```

5 Transfer

`Diaz/Transfer.lean`

Hmat_transfer [proved outright]

The transported matrix of a candidate is the matrix of its image: ‘Phi‘ carries ‘H u r‘ to ‘H t r‘ when it fixes ‘r‘ and sends ‘u‘ to ‘t‘.

```
theorem Hmat_transfer (Phi : C ->+* C) (hK : forall a in K, Phi a = a) {r : C} (hr : r in K)
 (hKconj : forall a in K, conj a in K) (hu0 : u != 0) (ht0 : t != 0) (hPhiu : Phi u = t)
 (hrho : u * conj u in K) (hrhot : t * conj t = u * conj u) :
 (Hmat u r).map Phi = Hmat t r
```

Uses: Hmat, conj_comm, self_mem_hull

coeff_transfer [proved outright]

A ring homomorphism fixing ‘K‘ carries the coefficient ‘wˆT M v‘ to the coefficient of the transported matrix, for ‘w, v‘ over ‘K‘. So vanishing of coefficients over ‘K‘ transfers.

```
theorem coeff_transfer (Phi : C ->+* C) (hK : forall a in K, Phi a = a)
 {m n : N} (M : Matrix (Fin m) (Fin n) C) (w : Fin m -> C) (v : Fin n -> C)
 (hw : forall i, w i in K) (hv : forall j, v j in K) :
 Phi (sum i, sum j, w i * M i j * v j) = sum i, sum j, w i * (Phi (M i j)) * v j
```

conj_comm [proved outright]

An isomorphism matching the generators intertwines conjugation. If ‘Phi‘ fixes ‘K‘ pointwise and sends ‘u‘ to ‘t‘, where ‘u‘ and ‘t‘ both lie on the circle ‘z * conj z = rho‘ over ‘K‘, then ‘Phi‘ commutes with complex conjugation on the whole hull of ‘u‘. Note where this comes from: ‘conj u = rho / u‘ is a *rational function of ‘u‘ over ‘K‘*, so conjugation is not extra data that could fail to be matched — it is already determined by the ring structure. That is the content of the closure theorem.

```
theorem conj_comm (Phi : C ->+* C) (hK : forall a in K, Phi a = a)
 (hKconj : forall a in K, conj a in K)
 (hu0 : u != 0) (ht0 : t != 0) (hPhiu : Phi u = t)
 (hrho : u * conj u in K) (hrhot : t * conj t = u * conj u) :
 forall z in hull K u, Phi (conj z) = conj (Phi z)
```

Uses: conj_eq_rho_div, eqOn_hull

exists_conj_intertwining [rests on an import]

The closure theorem, existence included. Any two candidates over the same base, with the same ‘rho‘, are carried onto one another by a ring endomorphism of ‘C‘ that fixes the base and intertwines conjugation. The intertwining is not arranged — it follows, because ‘conj u = rho/u‘ makes conjugation a rational function of the generator.

```
theorem exists_conj_intertwining (hKconj : forall a in K, conj a in K)
 (hu : Transcendental (K) u) (ht : Transcendental (K) t)
 (hrho : u * conj u in K) (hrhot : t * conj t = u * conj u) :
 exists Phi : C ->+* C, (forall a in K, Phi a = a) and Phi u = t
 and forall z in hull K u, Phi (conj z) = conj (Phi z)
```

Uses: conj_comm, exists_ringHom_of_transcendental, transcendental_ne_zero

hullsEquiv [proved outright]

The hulls of any two elements transcendental over ‘K‘ are isomorphic as ‘K‘-algebras, both being rational function fields.

```
noncomputable def hullsEquiv (hu : Transcendental (K) u) (ht : Transcendental (K) t) :
 (IntermediateField.adjoin (K) {u}) ~=a [K] (IntermediateField.adjoin (K) {t})
```

`injective_of_hom` [proved outright]

A ring homomorphism of fields is injective. That injectivity is what *would* give preservation of ‘K‘-linear independence, and hence of rank and structural rank; neither is formalized here, and this lemma states only the injectivity.

```
theorem injective_of_hom (Phi : C ->+* C) : Function.Injective Phi
```

6 The rank-one matrix

`Diaz/Rigidity.lean`

`Hmat` [proved outright]

The matrix ‘H‘ attached to a candidate.

```
noncomputable def Hmat (u r : C) : Matrix (Fin 2) (Fin 2) C
```

`candidate_no_vanishing_coeff` [rests on an import]

If ‘u‘ is a Diaz candidate over a base algebraic over ‘Q‘, then the attached rank-one matrix has no vanishing coefficient over that base.

```
theorem candidate_no_vanishing_coeff {L : Subfield C} [Algebra.IsAlgebraic Q (L)]
 {u r : C} (hu0 : u != 0) (hexp : IsAlgebraic Q (Complex.exp u))
 (hr : r in L) (hr0 : r != 0) (h : u * conj u = r ^ 2)
 (w v : Fin 2 -> C) (hwK : forall i, w i in L) (hvK : forall j, v j in L)
 (hw : w != 0) (hv : v != 0) :
 sum i, sum j, w i * (Hmat u r) i j * v j != 0
```

Uses: `no_vanishing_coeff_matrix`, `transcendental_candidate_over_base`

`coeff_eq_matrix` [proved outright]

```
theorem coeff_eq_matrix (u r : C) (w v : Fin 2 -> C) :
 sum i, sum j, w i * (Hmat u r) i j * v j
 = w 0 * (u * v 0 + r * v 1) + w 1 * (r * v 0 + conj u * v 1)
```

Uses: `Hmat`

`coeff_factor` [proved outright]

The coefficient ‘w^T H v‘ factors, because ‘H‘ has rank one.

```
theorem coeff_factor (hu0 : u != 0) (h : u * conj u = r ^ 2)
 (w v : Fin 2 -> C) :
 w 0 * (u * v 0 + r * v 1) + w 1 * (r * v 0 + conj u * v 1)
 = (w 0 + (r / u) * w 1) * (u * v 0 + r * v 1)
```

`det_Hmat` [proved outright]

‘det H = 0‘: over ‘C‘ the rows are dependent.

```
theorem det_Hmat (h : u * conj u = r ^ 2) : (Hmat u r).det = 0
```

Uses: `Hmat`

`mem_of_lin_rel` [proved outright]

If ‘u * a + r * b = 0‘ with ‘a, b‘ in ‘K‘, not both zero, and ‘r != 0‘ in ‘K‘, then ‘u in K‘.

```
theorem mem_of_lin_rel (hr : r in K) (hr0 : r != 0) {a b : C}
 (ha : a in K) (hb : b in K) (hab : not (a = 0 and b = 0))
 (h : u * a + r * b = 0) : u in K
```

`no_vanishing_coeff` [proved outright]

No vanishing coefficient. For non-zero ‘w, v‘ over ‘K‘, the coefficient ‘w^T H v‘ is non-zero.

```
theorem no_vanishing_coeff (hr : r in K) (huK : u notin K)
 (h : u * conj u = r ^ 2)
 (w v : Fin 2 -> C) (hwK : forall i, w i in K) (hvK : forall i, v i in K)
 (hw : w != 0) (hv : v != 0) :
 w 0 * (u * v 0 + r * v 1) + w 1 * (r * v 0 + conj u * v 1) != 0
```

Uses: `coeff_factor`, `mem_of_lin_rel`, `ne_zero_of_notMem`

`no_vanishing_coeff_matrix` [proved outright]

No vanishing coefficient, in matrix form.

```
theorem no_vanishing_coeff_matrix (hr : r in K) (huK : u notin K)
 (h : u * conj u = r ^ 2)
 (w v : Fin 2 -> C) (hwK : forall i, w i in K) (hvK : forall j, v j in K)
 (hw : w != 0) (hv : v != 0) :
 sum i, sum j, w i * (Hmat u r) i j * v j != 0
```

Uses: `coeff_eq_matrix`, `no_vanishing_coeff`

7 The intended base

`Diaz/Instantiation.lean`

`Qbar` [proved outright]

The algebraic numbers, as a subfield of ‘C‘.

```
noncomputable def Qbar : Subfield C
```

`QbarIsAlgebraic` [proved outright]

The instance that does not fire on its own. Named rather than anonymous so that it appears in the generated artefacts: an anonymous instance has no identifier for the generator to match, and would be invisible to the very document meant to expose the assumed surface.

```
noncomputable instance QbarIsAlgebraic : Algebra.IsAlgebraic Q (Qbar)
```

Uses: `Qbar`

`candidate_indistinguishable` [rests on an import]

A candidate is indistinguishable from an ordinary complex number. If ‘u‘ is a Diaz candidate over ‘Qbar‘ with ‘u * conj u = r^2‘, ‘r‘ real and algebraic, then there is a ‘t‘ — transcendental, on the same circle, carrying no algebraic exponential — and a ring endomorphism of ‘C‘ fixing ‘Qbar‘, sending ‘u‘ to ‘t‘, and intertwining conjugation on the hull of ‘u‘. This runs from the arithmetic hypotheses to the conclusion with no step left in prose.

```
theorem candidate_indistinguishable
 {u r : C} (hu0 : u != 0) (hexp : IsAlgebraic Q (Complex.exp u))
 (hr : r in Qbar) (hrr : conj r = r) (hr0 : r != 0)
 (h : u * conj u = r ^ 2) :
 exists t : C, t != 0 and Transcendental (Qbar) t and
 exists Phi : C ->+* C, (forall a in Qbar, Phi a = a) and Phi u = t
 and forall z in hull Qbar u, Phi (conj z) = conj (Phi z)
```

Uses: conj_mem_Qbar, exists_conj_intertwining, exists_transcendental_on_circle_sq, transcendental_candidate_over_ba

candidate_no_vanishing_coeff_Qbar [rests on an import]

The end-to-end statement over the intended base. If ‘u‘ is a Diaz candidate — non-zero, with ‘exp u‘ algebraic and ‘u * conj u = r^2‘ for an algebraic ‘r‘ — then the attached rank-one matrix has no vanishing coefficient over the algebraic numbers. This is ‘candidate_no_vanishing_coeff‘ with ‘L = Qbar‘, and its point is that the hypotheses are the arithmetic ones rather than transcendence, and that the base is the one the note is about rather than an arbitrary subfield.

```
theorem candidate_no_vanishing_coeff_Qbar
 {u r : C} (hu0 : u != 0) (hexp : IsAlgebraic Q (Complex.exp u))
 (hr : r in Qbar) (hr0 : r != 0) (h : u * conj u = r ^ 2)
 (w v : Fin 2 -> C) (hwK : forall i, w i in Qbar) (hvK : forall j, v j in Qbar)
 (hw : w != 0) (hv : v != 0) :
 sum i, sum j, w i * (Hmat u r) i j * v j != 0
```

Uses: Qbar, candidate_no_vanishing_coeff

conj_mem_Qbar [proved outright]

‘Qbar‘ is conjugation-stable, which every result above assumes of the base. If ‘p‘ kills ‘a‘ then it kills ‘conj a‘, its coefficients being rational and so fixed by conjugation.

```
theorem conj_mem_Qbar {a : C} (h : a in Qbar) : conj a in Qbar
```

Uses: mem_Qbar_iff

exists_transcendental_on_circle_Qbar [rests on an import]

And the circle carries a transcendental point over that base, so the model results are instantiated too.

```
theorem exists_transcendental_on_circle_Qbar {r : C} (hr : r in Qbar) (hr0 : r != 0) :
 exists t : C, t != 0 and Transcendental (Qbar) t and t * conj t = r * conj r
 and t * conj t in Qbar
```

Uses: conj_mem_Qbar, exists_transcendental_on_circle

mem_Qbar_iff [proved outright]

```
theorem mem_Qbar_iff {a : C} : a in Qbar <-> IsAlgebraic Q a
```

Uses: Qbar